Scalable App?

→ created an app → no. of people using less (response time → eg 2s)

↳ In future → Load ↑ (people ↑) → ↑ Response Time (Thus, not scalable)

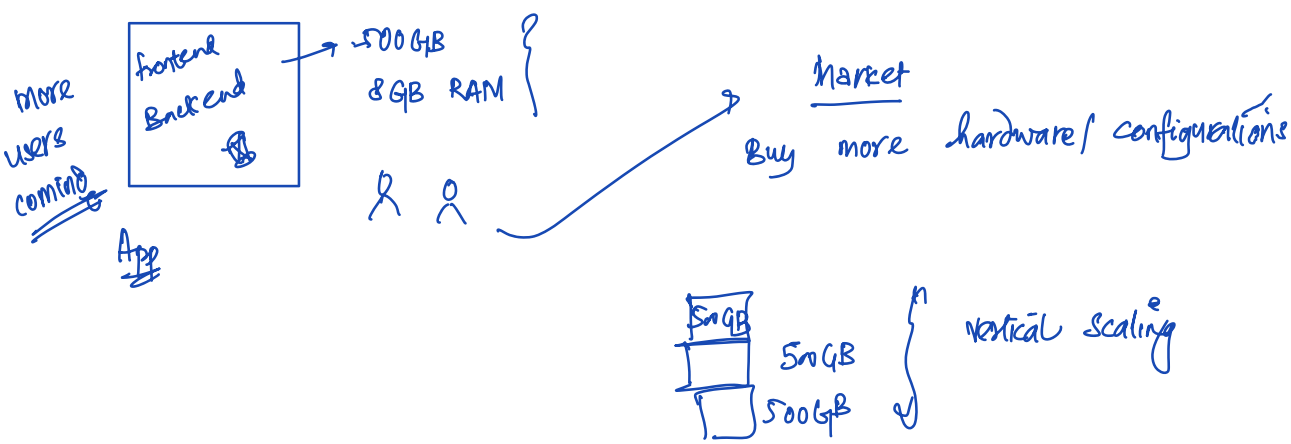
↳ Response Time maintained (scalable)

eg IRCTC
↓
Tatkaal Tickets → many people on it → Response Time ↓
(end)

↳ Not much scalable

eg Results on website → servers brought down → Not scalable.

Scale a monolithic application



- However, it has a limit - It is limited to capacity of one m/c, scaling beyond that capacity can involve down time and has an upper hard limit, ie, scale of hardware on which we're currently running

Limitations

① Single point of failure (all eggs in one basket)

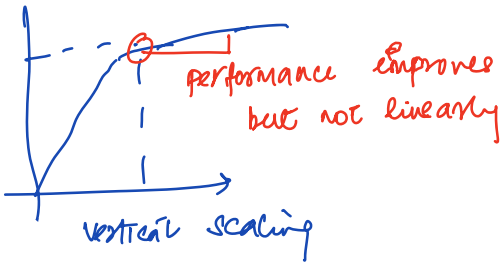
└ N/w failure etc

↓

Loose complete system

② After a point, it becomes very expensive.

└ Performance



- It is used in applications for small and medium sized applications which have limited users. Suppose there is exponential increase in number of clients, then it might not be possible to handle it after some point, so vertical scaling fails

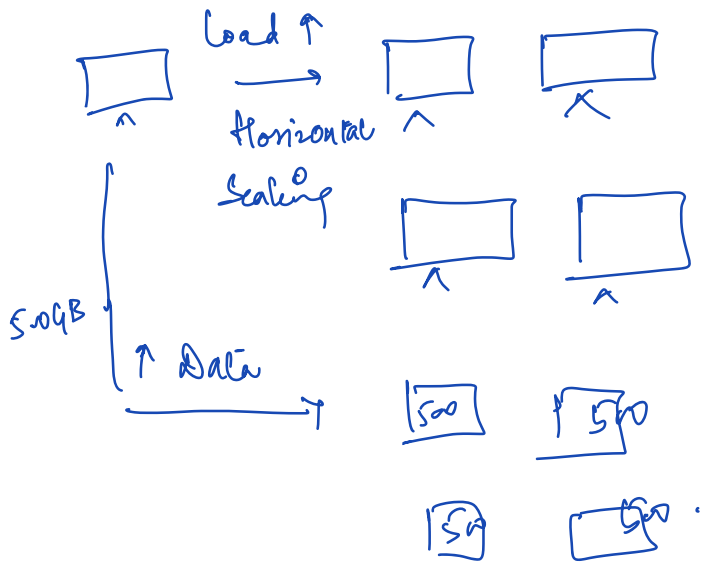
Horizontal Scaling

When load of application ↑

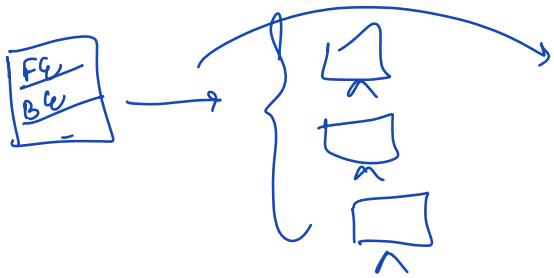
- Storage
- computation

When instead of single m/c – we take help of multiple m/cs – connected over hardware (These are commodity hardware (not very expensive m/cs)

Computation



- If it is monolithic app, can horizontal scaling be applied? → Yes



This monolithic app is distributed.

⇓

Distributed Monolithic App

Challenges of horizontal scaling of monolithic apps

① Maintaining consistency – Data replication

② No selective scaling

They all live together

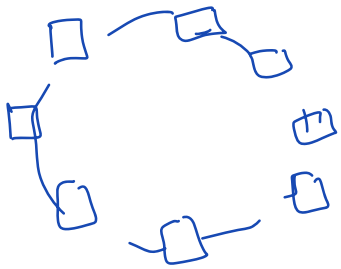
but we need to scale only BE

But here, we need to scale everything FE, DB too.

* However, horizontal scaling is diff^t from distributed systems →

Distributed Systems	Horizontal scaling.
→ They've diff^t components running on diff^t m/cs	→ Same components are instantiated and run on diff^t m/cs.
→ Components need to communicate through N/W calls.	→ M/c can complete whole task w/o any need to communicate with others.
~ A collection of independent computers that work together as a single logical unit (Eg) Team of people collaborating on one big project	→ Adding more M/cs to handle more load (Eg) Hiring more people to do same job

Replication & Redundancy



Handle failure gracefully

↓

Partition Tolerance/

failure Tolerance.

↓

Redundancy (Duplicates)

→ Used in context of nodes (computers/ components)

↳ Having duplicate copy of nodes, so that if node fails, other node is available to service customers

↳ Failure Recovery

↳ Fail Over Management

Types of Redundancy

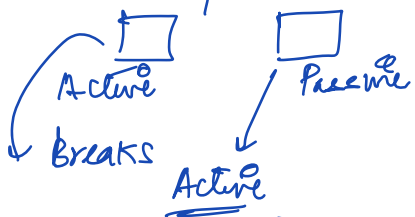
① Active Redundancy - each M/c is reacting to a request

Load Balancer

distributes load among all these components

② Passive Redundancy

(few nodes active / passive state)



when active breaks, passive becomes active

Reasons for partition in distributed system

(a) Absence of n/w

(b) Node going down

Replication (used in world of databases)

↳ Duplicating of data for multiple m/cs.

How its diff from redundancy?

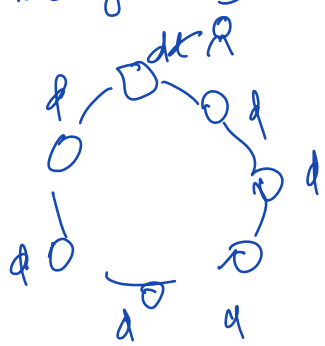
but redundancy is in world of m/cs.

Synchronising multiple m/cs.

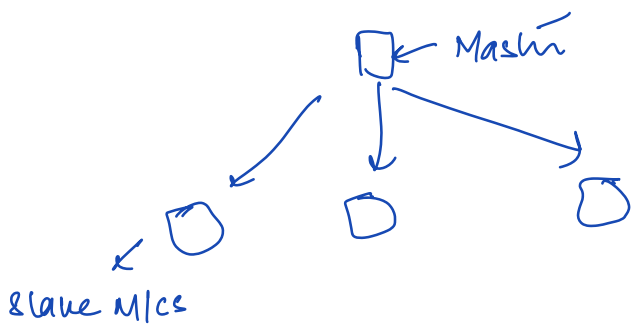
Types

(a) Active Replication

(Every node gets every data)



(b) Passive



(Every read/write on master →

all other m/cs are given this data →

but main read/write on master) — so if master goes down — one of slave m/cs become master

eg MySQL (RDBMS)

follow master - slave method

→ Redundancy → keeping multiple copies

→ Replication → Ensures all copies have same state via updates